Small Models' Hit Rate on Deciding Judgments, and VakilBuddy: A Citation-Grounded Workspace for Indian Advocates

Ayush Deshmukh
Pimpri Chinchwad College of Engineering, Pune
ayush.deshmukh23@pccoepune.org

Gargee Wagh
Pimpri Chinchwad College of Engineering, Pune
gargee.wagh23@pccoepune.org

Janhavi Dhamak
Pimpri Chinchwad College of Engineering, Pune
janhavi.dhamak23@pccoepune.org

Shreyas Bile
Pimpri Chinchwad College of Engineering, Pune
shreyas.bile23@pccoepune.org

Pradnya Narkhede
Pimpri Chinchwad College of Engineering, Pune
pradnya.narkhede@pccoepune.org

***Abstract*—Indian courts have begun to set aside decisions that relied on non-existent, AI-generated judgments. We measure how often small language models cite the judgment that decides a question, and build the open infrastructure and the advocate's workspace around that measurement. From the public AWS archive we build an audited index of 19,319,963 Indian judgments and orders, a citation verifier whose labels were correct in 96.8% of a held-out audit, and a citation graph of the Supreme Court with 109,370 links. Four open-weight models of 4 to 8 billion parameters, answering from memory, named a deciding judgment in 48 of 160 model–question cases, and only 40.0% of the cases they listed could be verified. Verifier feedback and search by party names removed most non-existent cases but found no better authorities. Search by subject over the reporters' headnotes, ranked with the graph's citation counts, raised the deciding-judgment hit rate to 133 of 160, with 98.5% of the listed cases verified. We then describe VakilBuddy, a prototype matter workspace that puts the index, a citation check, source checks on model-written text and an approval gate in front of an advocate, and show it on a real Supreme Court case. Existence checks catch fabrication; finding authority needs the text and the citation network.**

***Index Terms*—legal information retrieval, citation network, hallucination, citation verification, small language models, retrieval-augmented generation, Indian case law, legal workspace**

I. Introduction

Large language models (LLMs) now draft legal research notes, pleadings and even adjudicatory orders, and they cite authorities that do not exist. In 2023 a United States federal court sanctioned lawyers who had filed a brief citing decisions fabricated by ChatGPT [1]. The same failure has now reached adjudicators in India. In a ruling reported in October 2025, the Bombay High Court set aside an income-tax assessment of INR 27.91 crore that relied on non-existent, AI-generated case law [2]. In February 2026 the Supreme Court of India declared that a trial-court decision resting on "non-existent and fake alleged judgments" is "not an error in the decision-making process" but misconduct [3], and in September 2026 it set aside a customs penalty of INR 425.27 crore that relied on non-existent judgments and on real ones that did not say what was attributed to them [4].

Two different checks are needed: whether a cited judgment exists and the citation belongs to it, and whether it is an authority for the point, a judgment that decided the question rather than one that merely exists. For United States case law an open API performs the first check [5] and the citation network of the Supreme Court has long measured the importance of precedents [6]. For Indian case law the open sources are the Supreme Court Reports (S.C.R.), published digitally since 2024 [7], and a public archive of Supreme Court and High Court judgments on the AWS Open Data registry [8], [9]. We are not aware of an open citation checker built on them, of an open citation graph of the Supreme Court (the largest open release of the archive's text leaves it out of scope [10]), of a citation-level measurement of how LLMs cite Indian judgments [11], [12], or of an open workspace that puts such checks in an advocate's hands. This paper provides all four.

- **Index and verifier.** An audited PostgreSQL index of 19,319,963 judgments and orders, and a verifier for Supreme Court citations (§III–§IV).
- **Citation graph.** 109,370 links between 38,357 Supreme Court judgments, extracted from their text (§V).
- **Retrieval.** Headnote search ranked with the graph's citation counts (§VI).
- **Small-model benchmark.** Four open-weight models, five conditions, 403 items each (§VII).
- **VakilBuddy.** The architecture of a workspace that applies the index, the checks and an approval gate, shown on a real Supreme Court case (§IX–§XIII).

II. Related Work

Dahl et al. found hallucination rates of 58% (ChatGPT 4) to 88% (Llama 2) on verifiable questions about United States federal cases [13], and commercial retrieval-augmented legal

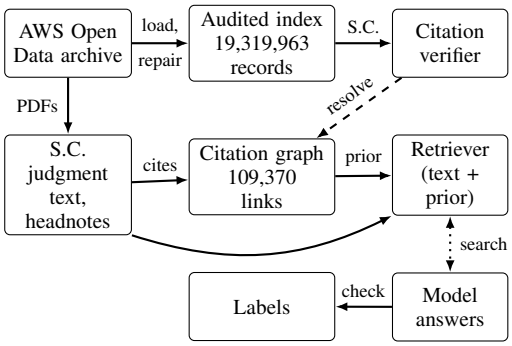


Fig. 1. Overview. The archive's metadata is loaded into an audited index; the verifier resolves citations and case names against its Supreme Court part. The text of every Supreme Court judgment yields headnotes and the citations each judgment makes, which the verifier resolves into a citation graph. The retriever searches headnotes by subject and ranks with the graph's citation counts. Model answers are checked by the verifier; in the search conditions the model queries the index or the retriever before answering.

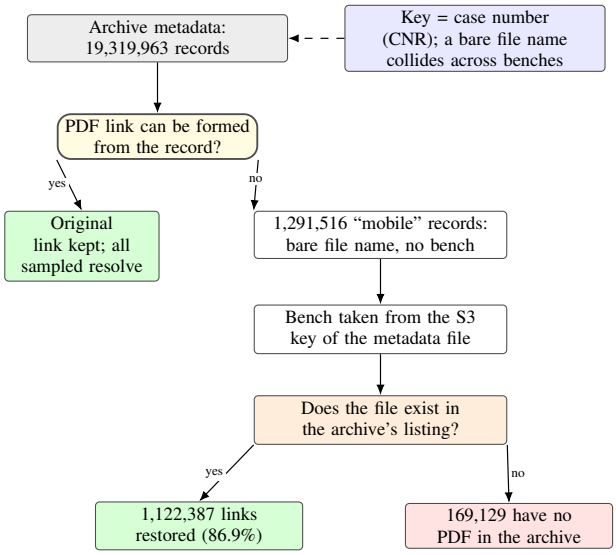


Fig. 2. Auditing and repairing the archive's High Court links (§III). Records are keyed on the case number, and a link is made only when its file exists in the archive's own listing.

tools still hallucinate on 17–33% of queries [14]; detector benchmarks [15], LegalCiteBench [16] and a study of substituted cases [17] also concern United States law. eyecite extracts United States citations [18] and CourtListener flags invalid ones [5]; our verifier plays that role for the Supreme Court of India, whose neutral and S.C.R. formats differ. Fowler et al. built the citation network of the United States Supreme Court [6]; prior-case retrieval takes its labels from the citations in judgments [19], [20], and retrieval-augmented generation grounds a model in retrieved documents [21]. Recent legal agents audit their citations [22]–[24]. Indian resources include ILDC [25], IL-TUR [26], NyayaRAG [27] and the Supreme Court's assistive tools SUVAS and SUPACE [28], [29]; open-india-law releases text and embeddings for 12.8 million judgments and treats a citation graph as out of scope [10].

III. An Open Index of Indian Judgments

The AWS Open Data registry hosts two CC-BY-4.0 datasets published by Dattam Labs: judgments of the Supreme Court from 1950 [8] and of 25 High Courts [9], with structured metadata in Parquet and the judgments as PDFs. We load only the metadata into one PostgreSQL 17 table of 19,319,963 records (38,366 Supreme Court judgments and 19,281,597 High Court judgments and orders, 26 courts, decided 1950–2026), each with a link to its PDF in the archive, never the PDF.

Audit and repair. Records from the archive's "mobile" path store a bare file name where a bench-qualified path is needed, so their PDF link cannot be formed [30]: 1,291,516 records in four High Courts (6.7% of the corpus), 97.8% decided in 2024–2026. The file name is no identity either: 61,125 names occur in more than one bench, covering 123,131 records, and in 88.3% of these the title and date both differ, so keying on the file name, as our first load did, silently overwrites at least 62,006 records. We key on the case number (CNR), take the bench from the S3 key, and link a record only when its file exists in the archive's listing. This restores 1,122,387 links (86.9%); the other 169,129 records have no PDF in the archive. After repair 99.1% of all records carry a link. All 300 sampled restored links, all 454 sampled original High Court links (24 courts) and all 150 sampled Supreme Court links resolve against the live archive, and for 450 sampled unrecoverable records the expected file does not exist.

Supreme Court metadata. Every one of the 38,366 judgments carries an S.C.R. citation and 38,238 a neutral citation [31]. Three properties matter for verification. The year field is the year of the S.C.R. volume, not of the decision, for 5,179 judgments (13.5%), so we use the decision date. 96 neutral citations are attached to more than one record. And for each year from 1950 to 2013 except 1961, 1994 and 1997 the index holds every neutral number from 1 to the year's maximum [32], so a missing number is unassigned for those 61 years.

IV. Citation Verifier

The verifier takes a case name, a year and a citation string, as a model produced them, and decides whether they denote a real Supreme Court judgment in the index (Fig. 3). Neutral and S.C.R. citations are looked up exactly (a citation without a volume matches any volume of that series, year and page); SCC, AIR and other reports cannot be resolved against the index, so for them the name decides. Names are compared word by word: stop-words are dropped, abbreviations expanded, and each word is weighted by $\log(N/\mathrm{df})$ over the N judgments [33]. Two words match if they are equal or, for words of five or more letters, if their similarity ratio [34] is at least 0.85, which accepts *Keshavananda*/*Kesavananda* but not *Patil*/*Patel*. The name score is the harmonic mean of the share of the cited name's weight that is matched (p) and of the judgment's (r), set to zero if $p < 0.8$ so that an unmatched given name cannot be outweighed by words such as *State*; a candidate must lie within a year of the stated year, and the threshold is

$\tau = 0.5$. VERIFIED establishes that the judgment exists near that year, not that it supports the proposition it is cited for. We developed the rules on the T4 answers of three models, reviewing a development sample of 56 decisions by hand, and then froze the verifier, recording hashes of its code, before the benchmark and the held-out audit of §VIII.

V. Citation Graph

We downloaded the PDF of every Supreme Court judgment and extracted its text layer with PyMuPDF (the reports carry an embedded OCR layer): 38,357 of 38,366 were extracted. Each judgment opens with the reporter's headnote, which we cut at the jurisdiction line and keep to 3,000 characters. We find every reporter citation (S.C.R., neutral, SCC, Supreme Court AIR) and resolve each distinct pair of citing judgment and citation in three tiers (Fig. 4A): the number, a parallel number written beside an SCC or AIR citation, and the name and year through the frozen verifier, accepted only if exactly one judgment matches. A pair whose number and name disagree is dropped. We developed the extractor on a pilot of 502 judgments and ran it once over the corpus.

It found 203,915 distinct pairs, of which 153,382 (75.2%) were resolved, giving 109,370 distinct links between 22,483 citing and 21,518 cited judgments: 56.1% of reported judgments are cited by a later one. Where a citation carries a number and a name that can both be resolved, the name and year alone reach the number's record in 97.1% of 46,091 cases, a label-free estimate of the precision of name-only links; another 1.3% reach a different record with matching parties, such as another order in the same case. Only 53 links (0.05%) point to a later judgment, which is impossible, and all 60 name-only links we drew for a hand check were correct. The most cited judgments are *Maneka Gandhi v. Union of India* (261 citing judgments), *Kesavananda Bharati* (170) and *Sharad Birdhi Chand Sarda* (155), and 50 of the 51 landmark judgments we chose before the graph existed are cited at least once.

VI. Retrieval

We index the title and headnote of every judgment two ways: BM25 [35] over unigrams and bigrams, and dense retrieval with the open multilingual model bge-m3 [36]. The hybrid retriever takes the union of the top 300 of each and adds their scores after normalising each; a citation prior adds the normalised $\log(1 + c)$, where c is the number of judgments in the graph that cite the candidate (Fig. 4B). Weights are chosen on a development split. The graph also supplies relevance labels without annotation [19], [20]: for each of 1,000 held-out judgments that cite at least 5 earlier ones, the query is its headnote with every citation and case name removed, the targets are the judgments it cites, and only earlier judgments and citations made before its date are used.

The hybrid retriever with the prior places 20.4% of the cited judgments in its top ten (hybrid alone 0.167, BM25 0.151, dense 0.136, popularity alone 0.013), a relative gain of 22% (paired bootstrap 95% CI of the difference $+0.030$ to $+0.043$). Searched with the question text, it returns a judgment that decided the question among its top three for 33 of the 40 legal questions (28 without the prior), while no model named such a judgment from memory for more than 24. The questions were written around landmark judgments, which favours the prior; the recommendation experiment has no such bias.

VII. How Small Models Cite

The benchmark uses 101 Supreme Court judgments (51 landmarks identified by name and 50 sampled at random, 1950–2023) and five tasks. T1 asks for the judgment reported at a citation (202 items); T2 asks the same for citations that cannot exist (60 items; declining is correct); T3 asks for the citations of a judgment given its parties and date (101 items); T4 asks for up to three leading authorities on each of 40 legal questions, each case labelled by the verifier; T5 gives the model the verifier's findings on its T4 answer and asks it to revise. For T4 we record reference answers, the judgments that decided each question (70 index records, including earlier decisions later overruled), compiled after the closed-book runs and before the search runs and hashed; a question counts as answered if a listed case matches one. We evaluate four open-weight models that fit an 8 GB laptop GPU, in 4-bit quantisation through Ollama [37]: Llama 3.1 8B [38], Qwen3 8B [39], Gemma 3 4B [40] and Qwen2.5-VL 7B [41], with one system prompt, temperature 0, a fixed seed and a JSON schema. In the name-search condition a model has two tools over the index metadata (search by party names, lookup by citation); in the two topic-search conditions its tool is the retriever, without and with the citation prior. Each condition was run once, after the code and the reference answers were frozen by hash.

VIII. Results: What Retrieval Changes

From memory. Llama and Gemma answered almost everything and were almost always wrong: Llama identified 1 of the 202 real citations, Gemma none, and both named a judgment for nearly every fabricated citation (59 and 60 of 60; Table II). The two Qwen models declined all 262 questions. No model identified more than 1 of 202 real citations, and given the parties and date they supplied 3 correct citations of 808 requested. Asked for authorities, the models cited between 102 and 117 cases each, of which only 23–61% could be verified (Table IV); they cited almost only the SCC and AIR reports, which the index cannot resolve, so those labels rest on party names and year. On a fresh stratified sample of 95 T4 decisions judged by hand the verifier was right in 92 (96.8%) , and of the 40 audited NOT_FOUND cases 27 named parties that match no Supreme Court judgment in any year.

Feedback and search by name. Telling a model what the verifier found removed the flagged cases (NOT_FOUND fell from 239 to 52; Table III) but added none: 147 of the 154 cases in the revised lists were repeated from the first answers. Name search almost eliminated cases the index cannot find (239 to 8), but fewer answers contained a deciding judgment (27 of 160 against 48): the models searched with the question's words,

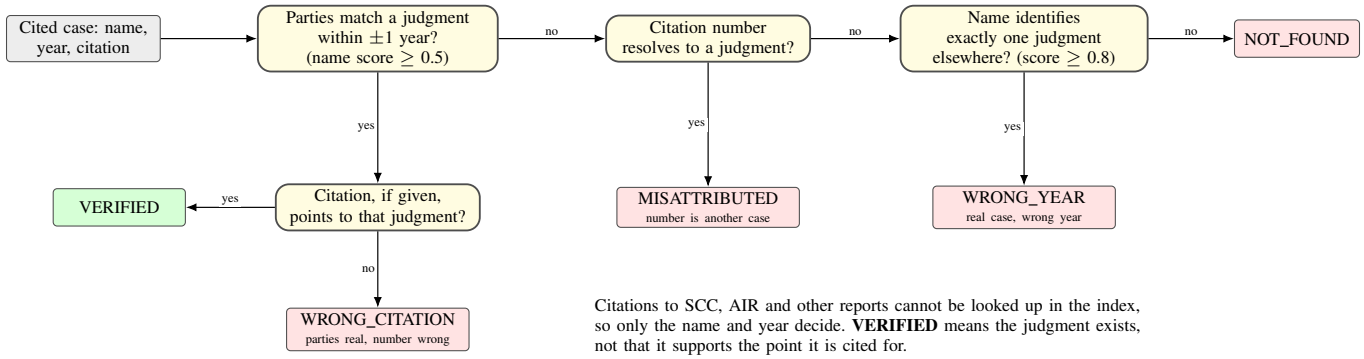


Fig. 3. How the verifier labels a citation (§IV). Each case a model cites ends in one of five labels; the audit of §VIII found 96.8% of them correct.

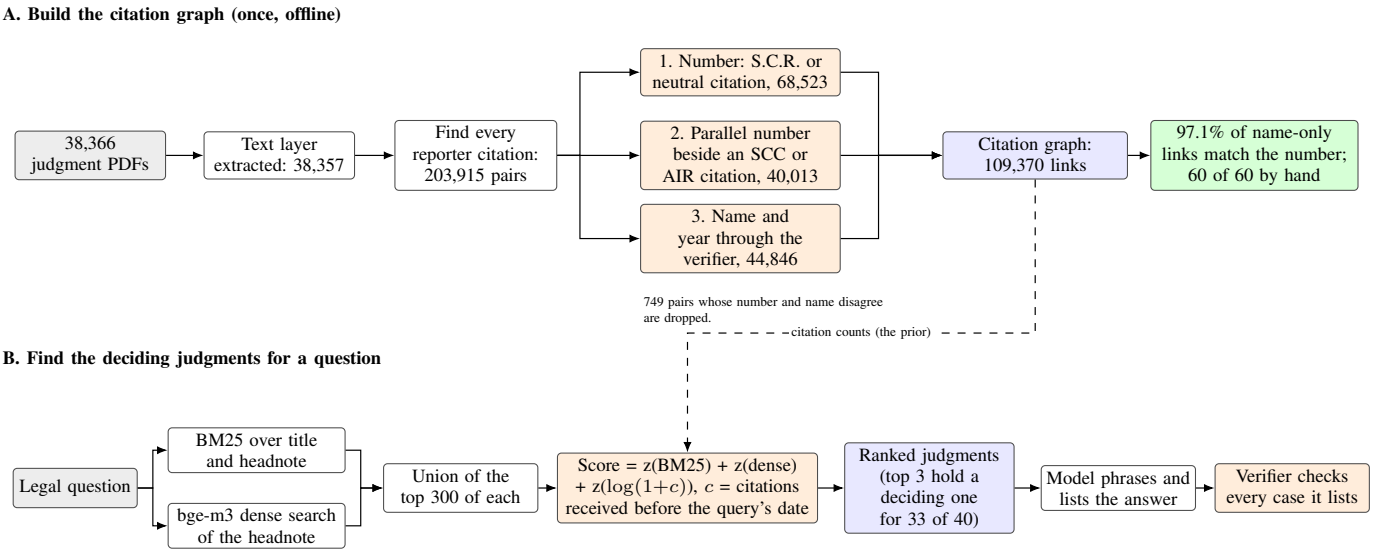


Fig. 4. From judgment text to checked authorities. (A) Every Supreme Court judgment is read for the citations it makes, and each is resolved in three tiers (§V). (B) A question is searched by subject, re-ranked with the graph's citation counts, and the model's answer is checked (§VI).

the tool matched party names, and the judgments it returned exist but decide nothing, such as *Fair Air Engineers v. N.K. Modi* for a question *Maneka Gandhi* answers.

Search by subject. With the retriever the models named a deciding judgment in 109 of 160 pairs, and in 133 with the citation prior (Fig. 6, Table I); counting each question once, 32 questions were answered by more models than from memory and 4 by fewer (exact sign test, $p < 0.001$). 98.5% of the 463 cases they listed were verified, against 40.0% from memory. The gain was largest for the model that knew least: Qwen2.5-VL named a deciding judgment for 3 questions from memory and 33 with the retriever. The models listed mostly what the search had shown them, and the retriever's top three alone answer 33 questions, about as many as the models do (32–35): the index guarantees existence, the retriever and the graph supply relevance, and the model at most phrases and orders the result.

Existence and authority are different checks. A verifier over names and citations catches fabricated cases, and feedback from it or search over it removed almost all of them, but it did not help the models find the judgment that decides a question, and name search made that rarer. A real but irrelevant judgment passes an existence check, so a system that shows only "verified" citations can look more reliable while citing worse authorities. Finding authority needs what the metadata lacks: what each judgment decided, which the headnote supplies, and which judgments the Court itself relies on, which the graph supplies. The prior favours old and

TABLE I
DECIDING JUDGMENTS NAMED AND CASES NOT FOUND, BY MODEL AND CONDITION (§VII). REF. IS THE NUMBER OF THE 40 QUESTIONS FOR WHICH A CASE THAT DECIDED THE QUESTION WAS NAMED; N.F. IS THE NUMBER OF LISTED CASES THE VERIFIER COULD NOT FIND.

	From memory		After feedback		Name search		Topic search		Topic + prior	
Model	Ref.	N.F.	Ref.	N.F.	Ref.	N.F.	Ref.	N.F.	Ref.	N.F.
Llama 3.1 8B	24	42	24	4	10	4	29	2	35	1
Qwen3 8B	13	49	13	1	12	1	24	0	33	0
Gemma 3 4B	8	81	8	16	5	1	27	0	32	0
Qwen2.5-VL 7B	3	67	3	31	0	2	29	0	33	0
All four	48	239	48	52	27	8	109	2	133	1

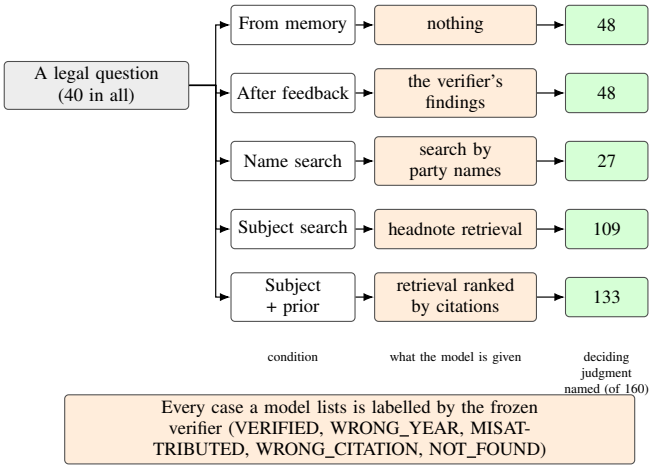


Fig. 5. The five conditions of the benchmark (§VII). Each of four models answered the same questions under each; the right-hand figure is the number of model–question pairs in which a case that decided the question was named.

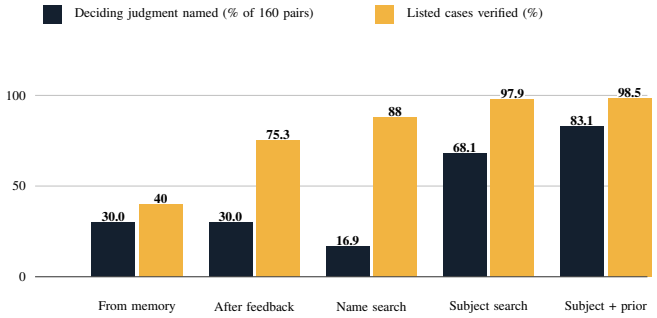


Fig. 6. What search and the citation graph change for four small open-weight models (§VIII). Existence checks (feedback, name search) raise the share of verified cases, but only search by subject, ranked with the citation prior, raises the share of questions answered with a deciding judgment.

TABLE II
IDENTIFYING A JUDGMENT FROM ITS CITATION. T1: 202 REAL CITATIONS. T2: 60 CITATIONS THAT CANNOT EXIST. CORR. = NAMED THE RIGHT JUDGMENT; NAMED = NAMED ANY JUDGMENT FOR A FABRICATED CITATION; DECL. = DECLINED; INV. = INVALID OUTPUT. "QWEN3 8B THINK" IS THE SAME MODEL WITH REASONING ENABLED.

	T1: real citation (%)				T2: fabricated (%)		
Model	Corr.	Wrong	Decl.	Inv.	Named	Decl.	Inv.
Llama 3.1 8B	0.5	99.0	0.0	0.5	98.3	0.0	1.7
Qwen3 8B	0.0	0.0	100.0	0.0	0.0	100.0	0.0
Qwen3 8B think	1.0	14.4	83.2	1.5	10.0	88.3	1.7
Gemma 3 4B	0.0	100.0	0.0	0.0	100.0	0.0	0.0
Qwen2.5-VL 7B	0.0	0.0	100.0	0.0	0.0	100.0	0.0

frequently cited judgments, including ones later overruled such as *A.D.M. Jabalpur*, which a lawyer must cite as overruled.

What remains unchecked. Neither the verifier nor the retriever checks that a judgment supports the proposition it is cited for, which needs its reasoning and is hard even for strong models [17], and the graph records that one judgment cites another, not whether it follows or overrules it. The graph and verifier cover the Supreme Court only: the High Court records have no citation fields (none of 5,592 sampled records had one), so extracting their citations needs the judgments themselves, about 1.7 TB of PDFs by our sample, which we did not download.

Limits of the benchmark. We evaluated four open-weight models of 4 to 8 billion parameters in 4-bit quantisation with one prompt and greedy decoding; larger and hosted models, including those VakilBuddy calls through OpenRouter, may behave differently and are untested. The benchmark is small (101 judgments, 40 questions), and the audit labels, reference answers and hand checks come from a single annotator, so we cannot report inter-annotator agreement; the reference answers were compiled after the closed-book runs and, being landmarks, favour the citation prior.

TABLE III
T5: THE T4 ANSWERS THAT CONTAINED AN UNVERIFIED CASE (Q), BEFORE AND AFTER THE MODEL REVISED THEM WITH THE VERIFIER'S FEEDBACK.

		Cases		Verified (%)		Not found	
Model	Q	Before	After	Before	After	Before	After
Llama 3.1 8B	30	87	48	49.4	87.5	42	4
Qwen3 8B	29	84	27	30.9	92.6	49	1
Gemma 3 4B	37	111	40	19.8	52.5	81	16
Qwen2.5-VL 7B	35	87	39	9.2	20.5	67	31

TABLE IV
T4: EVERY CASE CITED FOR 40 LEGAL QUESTIONS, BY VERIFIER LABEL. VER. = VERIFIED (WITH WILSON 95% INTERVAL), W.YR. = WRONG_YEAR, W.CIT. = WRONG_CITATION, MISATT. = MISATTRIBUTED, N.F. = NOT_FOUND.

		Share of cited cases (%)					
Model	Cases	Ver.	95% CI	W.Yr.	W.Cit.	Misatt.	N.F.
Llama 3.1 8B	114	61.4	52–70	1.8	0.0	0.0	36.8
Qwen3 8B	117	50.4	42–59	7.7	0.0	0.0	41.9
Qwen3 8B think	94	35.1	26–45	4.3	0.0	0.0	60.6
Gemma 3 4B	117	23.9	17–32	6.8	0.0	0.0	69.2
Qwen2.5-VL 7B	102	22.5	16–32	0.0	0.0	11.8	65.7

IX. VakilBuddy: Architecture

The findings above answer three weaknesses of ordinary legal assistants: single-turn answers, similarity without authority, and no independent check. They give three design principles. *The index, not the model, decides what exists*: every citation is looked up and model text is checked against sources before it is shown. *Authority comes from the record*: a draft may cite only the judgments the advocate pinned, and the citation graph, not text similarity alone, ranks what to pin. *The advocate decides*: work moves through stages with an approval gate, and the workspace files and sends nothing.

VakilBuddy is a prototype litigation workspace organised around a *matter* (a case file); Fig. 7 shows its parts. The workspace runs in the advocate's browser and keeps a matter's data in browser storage. A small server serves the page, builds model prompts, applies the source checks and holds the model key, which is never returned to the browser. Model requests go to OpenRouter [42] or any OpenAI-compatible endpoint. A separate case-law API answers searches and exact lookups over the index of §III. A court-status service, with a cache, per-customer keys, rate limits and a markup-drift check, answers from saved sample pages in this prototype. The verifier, graph and retriever of §IV–§VI are research code on the same database; the workspace does not call them yet and uses a simpler lookup (§XI). Table V lists what the workspace does today. Features marked "model" call a language model and pass its output through the checks of §XI; the others are deterministic.

X. Workflow and the Approval Gate

A matter moves through the stages of Fig. 8. Intake turns the advocate's account into a matter, research pins authorities, evidence is read and a chronology built, and arguments link points to authorities. Drafting starts from the matter's own record: pinned authorities, chronology and documents flow into drafts (Fig. 9a), so a fact or an authority is entered once. A draft is either *Draft* or *Approved* (Fig. 9b). Approval warns if review comments are open, saves a version and writes an audit-trail entry; an approved draft is locked, and a Word file or a print can be made only from an approved draft. Versions (the latest twelve) are saved when a draft is made, before an AI draft replaces text, when the advocate saves and on approval, and a word-level redline compares any two. This is the duty of care an advocate already owes the court [3]: a flagged citation is corrected by the advocate and never rewritten automatically.

XI. Checks on Citations and on Model Text

The citation check (Fig. 10A) uses no model. It finds citations and case names by pattern, merges parallel citations, asks the index for each number and, failing that, searches by name with state abbreviations expanded ("M.P." and "Madhya Pradesh") and a one-letter spelling slip tolerated. It is simpler than the verifier of §IV, and, like it, confirms existence and not that a judgment supports the proposition.

A model action (Fig. 10B) passes the advocate's text to the model as data and never as instructions. The reply must be valid JSON; a failed call is shown as an error and never replaced by a canned answer, and the percentages a model may offer for risk are not used. The reply is cleaned and then checked against sources. The cases it names are looked up in the index. A position attributed to a witness is kept only if its content words and every date and number in it appear in that witness's statement. A next hearing date is kept only if the order writes it. A chronology's source, and a document a witness is to be confronted with, must be documents that were supplied, and a dated event is kept only if the documents write its date. Items that fail are dropped and counted, the answer carries the model's name, and the advocate decides.

XII. Evidence, Dates and Drafts

Evidence (Fig. 11a) is read in the browser: text, Word and PDF files with a text layer directly, with pdf.js [43], and scans and photographs by Tesseract [44] in English, Hindi and Marathi. The files are not uploaded, and only their text is kept. Dates and rupee amounts, including Devanagari digits and month names, are found by pattern together with the nearest event word, and two documents that give different dates or amounts beside the same word are listed as possible inconsistencies: places to look, never findings. **Dates** (Fig. 11b): the calculator holds 27 periods, each naming its source (the Limitation Act, 1963 [45], the Supreme Court Rules, 2013 [46], the Code of Civil Procedure and others). It leaves out the day of the event (s. 12(1)) and, where the law allows it, the time taken to get a certified copy (s. 12(2)), runs a period that ends on a closed day to the day the court reopens (s. 4), and shows how the date was counted. **Drafts**: Manual fills one of 10 formats from the record and leaves a visible blank where the record has nothing. Auto (Fig. 11c) asks a model to write each section from the advocate's account and the record, under a prompt that allows only pinned authorities, only the matter's documents as annexures and a blank for a missing fact; the text is cleaned, every citation in it is looked up, and the earlier text is saved first as a version. A para-wise reply numbers the other side's paragraphs and builds the reply from a stance for each (admit, deny, partly, no knowledge, legal), leaving the substance to the advocate.

XIII. A Worked Case

We ran the workspace on a real Supreme Court case, *Rajendra S/o Ramdas Kolhe v. State of Maharashtra* (Criminal Appeal No. 2281 of 2011, decided 15 May 2024, 2024 INSC 422), a murder appeal that turns on a dying declaration. The judgment, the Court's summaries of the six prosecution witnesses, the orders it recites and a draft written for this test are in `sample-case/`; the scripts that produce every number below are in `paper/final/case_study/`.

Reading and dates. The Evidence tab read the judgment (8,619 words) and found 30 dates and 1 amount; "Ex. 59", the deceased's written dying declaration, occurs 23 times in it and 3 in the witness summaries (2,092 words). The three documents give no conflicting dates or amounts, so the list of possible inconsistencies is empty. For a review petition

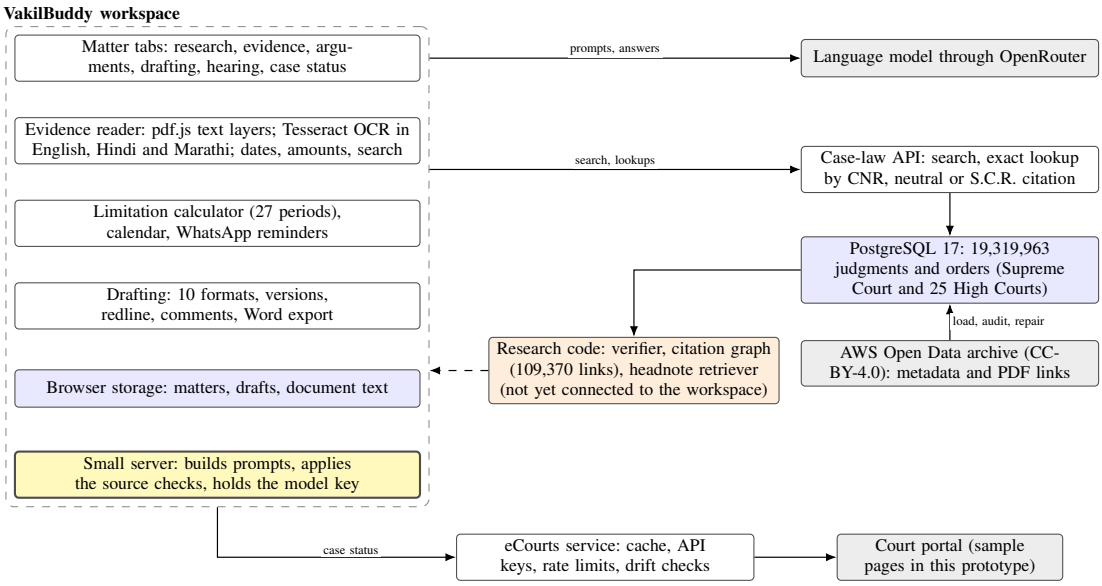


Fig. 7. Architecture of VakilBuddy as built. The workspace runs in the advocate's browser with a small server; the case-law API serves the index of §III; model requests go to OpenRouter; the court-status service answers from sample pages. The verifier, graph and retriever of §IV–§VI are research code that the workspace does not yet call.

TABLE V
WHAT THE WORKSPACE DOES TODAY, AND WHERE A LANGUAGE MODEL IS INVOLVED.

Feature	What it does	Built from	Model
Case-law search	Exact lookup by CNR, neutral or S.C.R. citation; full-text and typo-tolerant search; filters by court, year and outcome	the index (§III)	no
Citation check and pinned authorities	Finds the citations and case names in a text and looks each up; pins the judgments a matter relies on, each with a note of what for	the index	no
Evidence reading	Text, Word and PDF text layers; OCR in English, Hindi and Marathi; dates, amounts, search, possible inconsistencies	pdf.js, Tesseract	no
Chronology, cross-examination questions, witness comparison	Dated events and questions from the documents; positions that differ between witnesses	the matter's documents	model
Arguments	Points linked to pinned authorities; the other side's best answer to each	pinned authorities	model
Drafting	10 formats filled from the record (Manual); sections written by a model that may cite only pinned authorities (Auto)	the record, pinned authorities	Auto
Para-wise reply	Numbers the other side's paragraphs; builds the reply from a stance for each, leaving the substance blank	the plaint or notice	no
Review and export	Versions, word-level redline, comments, approval, Word export with court-rule profiles	drafts	no
Limitation and reminders	27 periods with their sources; counting by s. 12 and s. 4; saved dates; WhatsApp text the advocate sends	the statutes	no
Court status	Case record by CNR; an order digest; preparation for the next listing	the court-status service (sample data)	digest

(Order XLVII Rule 2, Supreme Court Rules, 2013 [46]) the calculator, given 15 May 2024, returns 14 June 2024 and shows that the day of the judgment is left out.

Checking a draft. The draft is a few paragraphs of submissions into which we planted errors (Table VI). The citation check found 8 cases. Of these, 4 were confirmed. One has the right parties and a wrong volume (*Paniben*, cited as [1992] 9 S.C.R. 197, reported at [1992] 2 S.C.R. 197). One gives the appellant's own case the number of *Khushal Rao v. State of Bombay*. One, *Verma v. State of Karnataka*, does not exist. One is a High Court case, reported as not checkable. We planted the errors, so this shows that the check catches those kinds of error, not how often they occur. The case also exposed three defects in how the check searches the index (a state written in full where the archive writes "M.P.", a party name matched to a different case, and a spelling slip in a title); we fixed them and added regression tests. One limit remains: *Sudhakar v. State of Madhya Pradesh*, which the archive spells "Shudhakar", is found by its number but not by name, because the index is searched by whole words.

Checks on model text. We sent the real prompts of three model actions that read these documents to Qwen2.5-VL 7B, one of the four models of §VII, three times each (temperature 0.2, seeds 1–3, a context of 8,192 tokens), kept its raw answers,

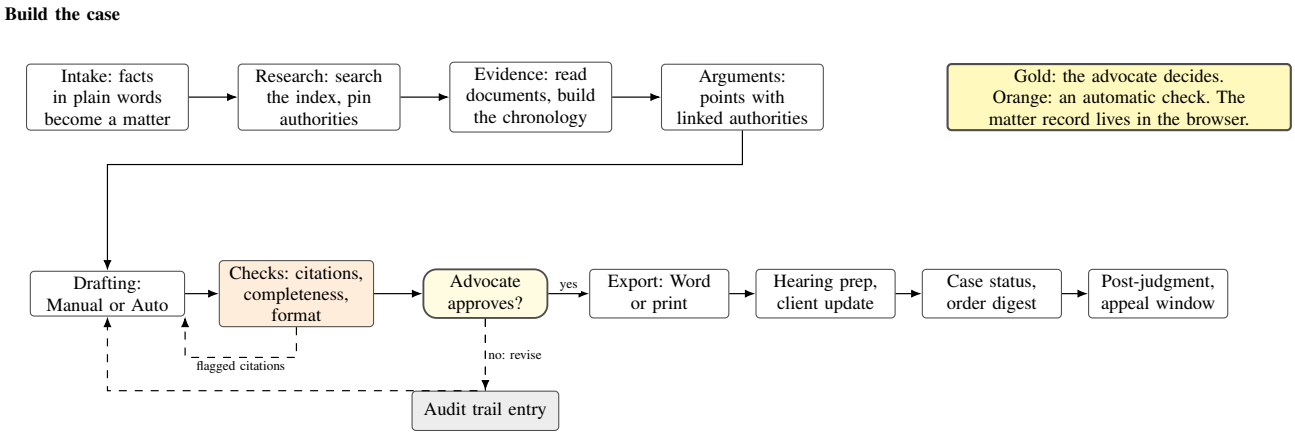


Fig. 8. The life of a matter in VakilBuddy. Nothing leaves the workspace until the advocate approves a draft, and an approval is recorded in the audit trail. A flagged citation is fixed by the advocate; there is no automatic rewriting loop.

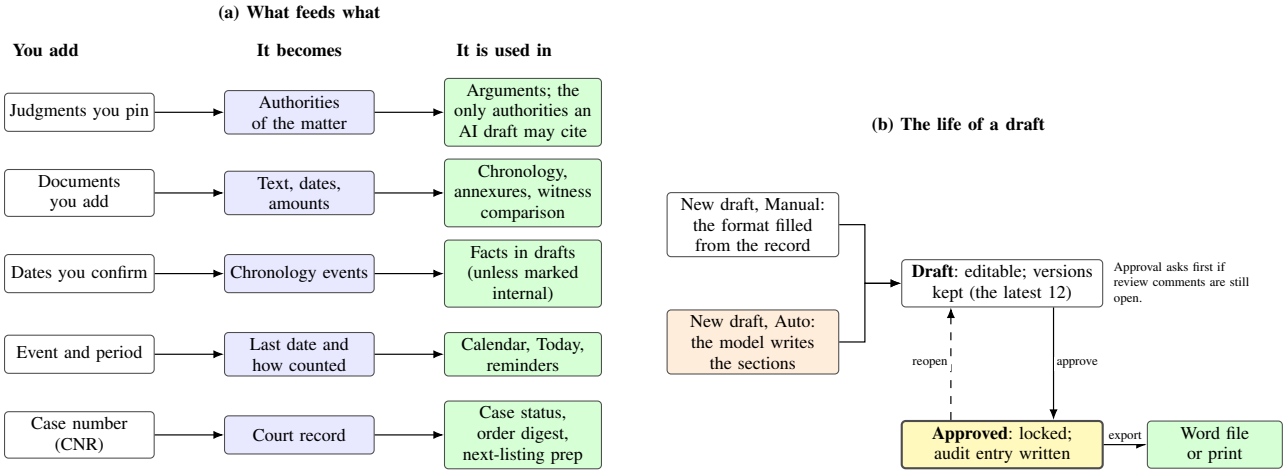


Fig. 9. (a) How the matter tabs feed each other: pinned authorities, confirmed dates and read documents flow into arguments and drafts. (b) A draft can be exported only after the advocate approves it; an approved draft is locked until reopened.

TABLE VI
THE DRAFT WITH PLANTED ERRORS AND WHAT THE CITATION CHECK REPORTED.

Case as cited	Citation	Verdict
Khushal Rao v. State of Bombay	[1958] 1 S.C.R. 552 · 1957 INSC 79	Confirmed
Ashabai v. State of Maharashtra	[2013] 1 S.C.R. 115	Confirmed
Lakhan v. State of Madhya Pradesh	[2010] 9 S.C.R. 705 · 2010 INSC 489	Confirmed
Paniben (Smt.) v. State of Gujarat	[1992] 9 S.C.R. 197	Wrong number
Sher Singh v. State of Punjab	2008 INSC 204	Confirmed
Rajendra Kolhe v. State of Maharashtra	[1958] 1 S.C.R. 552	Number of another case
Verma v. State of Karnataka	[2011] 5 S.C.R. 410	Not found
State v. Rao	(name only)	High Court, not checkable

and counted what the source checks removed. The witness comparison returned 4 points in 3 runs (1 run returned none); the checks dropped 1, a point whose second side named no witness, and kept 3, which quote the statements faithfully. The checks test fidelity to the source, not whether two statements conflict, and whether a kept point is a contradiction an advocate could use is a legal judgment we do not make. In all 3 runs the order digest gave a next hearing date, 30 May 2024 (2 runs) and 29 May 2024, worked out from "two weeks from today"; the order writes none, and each was replaced by "not stated in the order". The chronology proposed 13 dated events, and 5 of 13 dates were not in the documents, so their events were dropped. The model named 13 sources and 12 documents to confront a witness with, and every one was a document we had supplied. The context had to be 8,192 tokens: at Ollama's default of 4,096 the chronology prompt, of about 4,000 tokens, left room for only 100 tokens of answer. This is one case, one small model and three runs per action; it shows that the checks remove what the documents do not support, not how often a larger or hosted model would produce it.

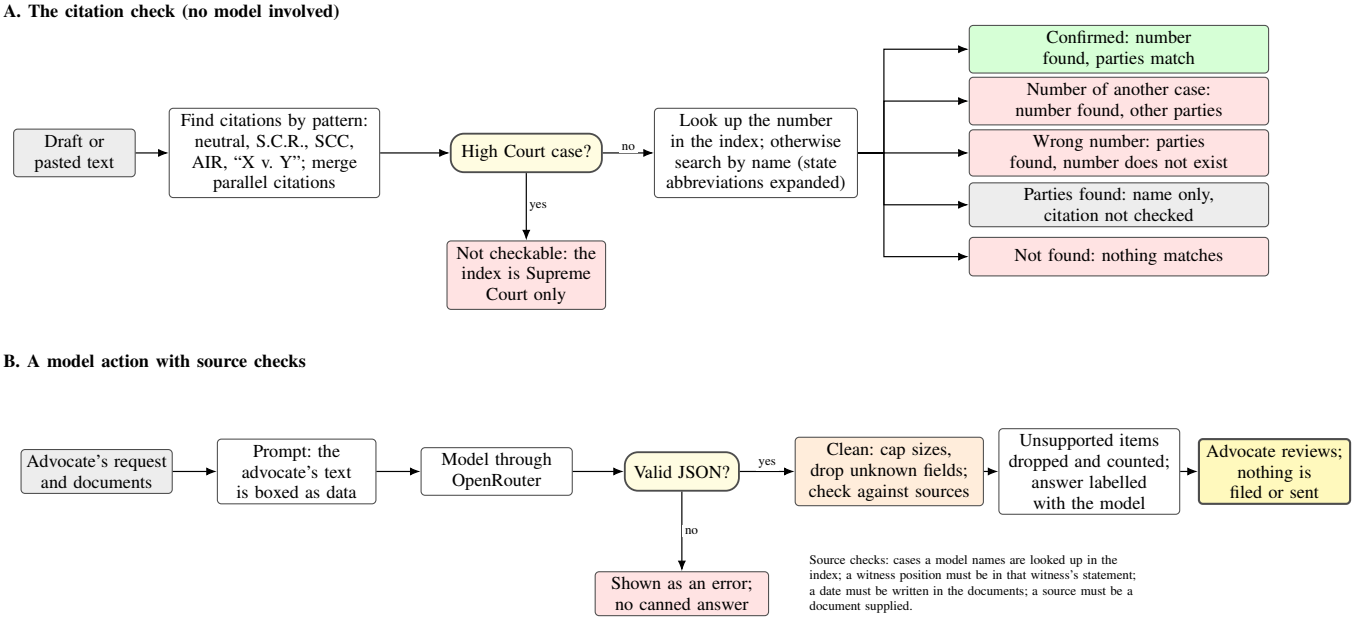


Fig. 10. The two check pipelines of the workspace. (A) The citation check uses patterns and the index, never a model. (B) Every model action passes the advocate's text to the model as data, and what comes back is cleaned and checked against the advocate's own documents and the index before it is shown (§XI).

XIV. Limitations and Ethics

Limitations. VakilBuddy is a prototype: its matters other than the worked case are sample data, its court-status service answers from sample pages, its citation check is simpler than the evaluated verifier, and the workspace is covered by 206 automated tests and its index service by 38, not by a user study, so we make no claim about drafting quality or time saved. On one laptop, exact lookups in the index take a median of 2.0 ms and the verifier 80 ms.

Ethics. The workspace is decision support: the advocate remains responsible to the court, nothing is filed or sent by the workspace, every model answer is labelled and every draft needs approval. Evidence files are read in the browser and not uploaded, but the text of the documents an advocate sends to a model action goes to the model provider, which matters for privileged material. An AI assistant was used to build the software and draft this paper, as the acknowledgment states.

XV. Conclusion

Indian courts now treat reliance on fabricated precedents as misconduct [3], so a model-drafted citation must be checked before it reaches a court. From open data we built an audited index of 19,319,963 judgments and orders, a verifier whose labels were correct in 96.8% of a held-out audit, and an open citation graph of 109,370 links. Small open models answering from memory fabricate freely; an existence check removes the fabricated cases but not the lack of authority; search by subject over headnotes, ranked with the graph's citation counts, raised the share of questions answered with a deciding judgment from 30% to 83% while 98.5% of the listed cases were verified. VakilBuddy applies these checks and an approval gate in an advocate's workspace, and a worked Supreme Court case shows them catching planted citation errors and unsupported model output. Connecting the evaluated retriever to the workspace, benchmarking the hosted models it calls, and measuring what it saves an advocate remain open.

Acknowledgment

An AI assistant (Claude, Anthropic) was used to write the software, run the experiments and analyses, produce the first-pass labels of the verifier audit, the manual check of the citation graph and the reference answers for T4, and draft the text. The authors are responsible for the content.

Data and Code Availability

The index builder, repair scripts, verifier, citation extractor, citation graph (every resolved and unresolved citation with its resolution tier), headnote embeddings, retrieval code, benchmark items, prompts, raw model outputs, scoring code, the workspace and the worked-case scripts and files are available from the authors. The underlying data are CC-BY-4.0 [8], [9].

References

[1] United States District Court for the Southern District of New York, "Mata v. Avianca, Inc., 678 F. Supp. 3d 443 (S.D.N.Y. 2023)," 2023, opinion and order on sanctions, June 22, 2023.

[2] Bar & Bench, "Bombay High Court pulls up tax officer for citing fake, AI-generated case laws (KMG Wires Private Limited v. AO, NFAC)," https://www.barandbench.com/news/litigation/bombay-high-court-pulls-up-tax-officer-for-citing-fake-ai-generated-case-laws, Oct. 2025, published Oct. 30, 2025; accessed Sep. 29, 2026.

[3] MediaNama, "SC says citing AI-generated fake case laws is misconduct (Gummadi Usha Rani v. Sure Mallikarjuna Rao, Supreme Court of India, order of Feb. 27, 2026)," https://www.medianama.com/2026/03/223-supreme-court-ai-fake-case-laws-misconduct/, Mar. 2026, accessed Sep. 29, 2026.

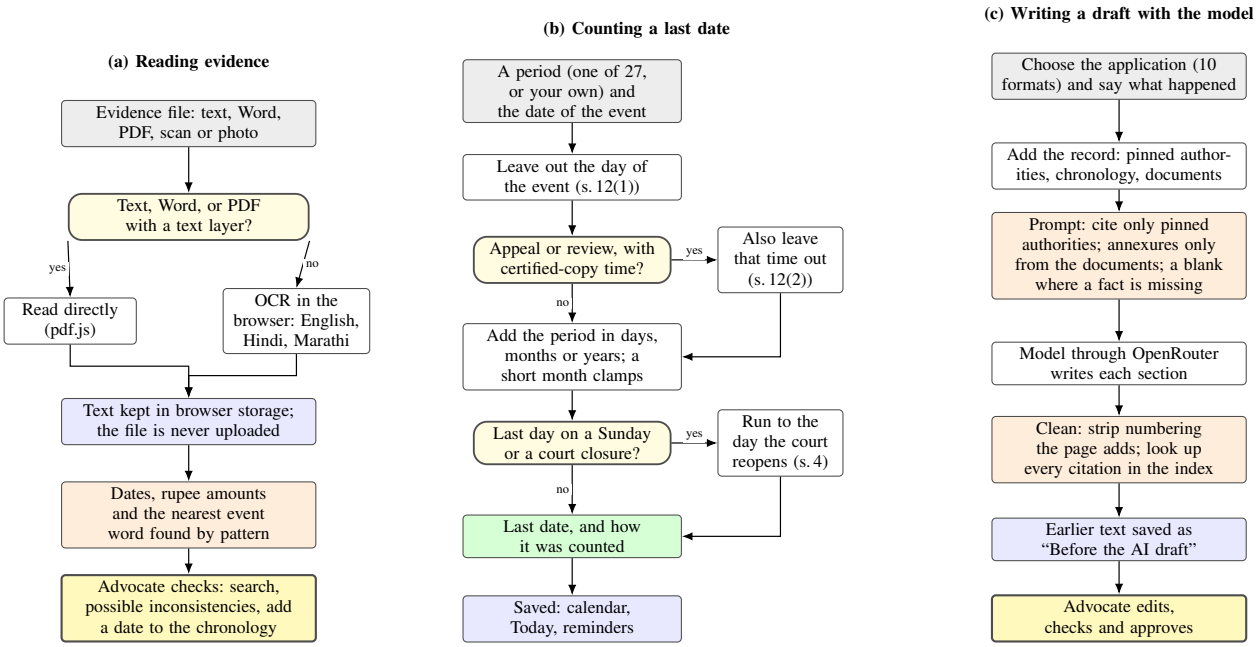


Fig. 11. Three processes of the workspace. (a) Evidence is read where it lies and only its text is kept. (b) A limitation date is counted by stated rules and shows its steps (§IX). (c) An AI draft is bound to the matter's own record and is a draft for the advocate, not a filing.

[4] SCC Online, "Supreme Court sets aside Rs. 425.27-crore customs penalty order built on AI-generated fake citations (Vijay Ghanshyam Gadiya v. Union of India, 2026 SCC OnLine SC 1741, decided Sep. 2, 2026)," https://www.scconline.com/blog/post/2026/09/03/supreme-court-sets-aside-%E2%82%B9425-crore-customs-penalty-over-ai-generated-fake-citations/, Sep. 2026, accessed Sep. 29, 2026.

[5] Free Law Project, "Combat hallucinations and look up citations with our new API," https://free.law/2024/04/16/citation-lookup-api/, Apr. 2024, accessed Sep. 29, 2026.

[6] J. H. Fowler, T. R. Johnson, J. F. S. II, S. Jeon, and P. J. Wahlbeck, "Network analysis and the law: Measuring the legal importance of precedents at the U.S. Supreme Court," *Political Analysis*, vol. 15, no. 3, pp. 324–346, 2007.

[7] Supreme Court of India, "Digital Supreme Court Reports (Digi-SCR)," https://digiscr.sci.gov.in/, accessed Sep. 29, 2026.

[8] Dattam Labs, "Indian Supreme Court judgments," Registry of Open Data on AWS, https://registry.opendata.aws/indian-supreme-court-judgments/, s3 bucket indian-supreme-court-judgments (ap-south-1), CC-BY-4.0; accessed Sep. 29, 2026.

[9] ——, "Indian High Court judgments," Registry of Open Data on AWS, https://registry.opendata.aws/indian-high-court-judgments/, s3 bucket indian-high-court-judgments (ap-south-1), CC-BY-4.0; accessed Sep. 29, 2026.

[10] Vaquill AI, "open-india-law: Open, structured Indian primary law," https://github.com/Vaquill-AI/open-india-law, snapshot v2026.08.1; accessed Sep. 29, 2026.

[11] R. Hemrajani, "Evaluating the role of large language models in legal practice in India," arXiv:2508.09713, 2025.

[12] K. Juvekar, A. Bhattacharya, S. Khadloya, and U. Saxena, "Are LLMs court-ready? Evaluating frontier models on Indian legal reasoning," arXiv:2510.17900, 2025.

[13] M. Dahl, V. Magesh, M. Suzgun, and D. E. Ho, "Large legal fictions: Profiling legal hallucinations in large language models," *Journal of Legal Analysis*, vol. 16, no. 1, pp. 64–93, 2024.

[14] V. Magesh, F. Surani, M. Dahl, M. Suzgun, C. D. Manning, and D. E. Ho, "Hallucination-free? Assessing the reliability of leading AI legal research tools," *Journal of Empirical Legal Studies*, vol. 22, no. 2, pp. 216–242, 2025.

[15] P. Liu, D. Stammbach, and P. Henderson, "Who checks the citations? Benchmarking legal hallucination detection," arXiv:2606.21155, 2026.

[16] S. Chen, H. Yin, and S. Zhou, "LegalCiteBench: Evaluating citation reliability in legal language models," arXiv:2605.10186, 2026.

[17] A. Verma, "Is this citation on point?" arXiv:2608.12571, 2026.

[18] J. Cushman, M. Dahl, and M. Lissner, "eyecite: A tool for parsing legal citations," *Journal of Open Source Software*, vol. 6, no. 66, p. 3617, 2021.

[19] A. Joshi, A. Sharma, S. K. Tanikella, and A. Modi, "U-CREAT: Unsupervised case retrieval using events extraction," in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2023, pp. 13 899–13 915.

[20] M. Färber and A. Jatowt, "Citation recommendation: approaches and datasets," *International Journal on Digital Libraries*, vol. 21, pp. 375–405, 2020.

[21] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. tau Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474.

[22] B. Yuan and Z. Wang, "L-MARS: Legal multi-agent system with agentic search and citation-faithfulness audit," in *Proceedings of the AI4Law Workshop at the 43rd International Conference on Machine Learning*, 2026, arXiv:2509.00761.

[23] H. Geng and L. Liu, "Parthenon Law: A self-evolving legal-agent framework," arXiv:2606.04602, 2026.

[24] B. Mounir, F. Madkour, A. Abdelaziz, and A. Sami, "Maat: The agentic legal research assistant for competition protection," arXiv:2605.27331, 2026.

[25] V. Malik, R. Sanjay, S. K. Nigam, K. Ghosh, S. K. Guha, A. Bhattacharya, and A. Modi, "ILDC for CJPE: Indian legal documents corpus for court judgment prediction and explanation," in *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, 2021, pp. 4046–4062.

[26] A. Joshi, S. Paul, A. Sharma, P. Goyal, S. Ghosh, and A. Modi, "IL-TUR: Benchmark for Indian legal text understanding and reasoning," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Bangkok, Thailand, 2024, pp. 11 460–11 499.

[27] S. K. Nigam, B. D. Patnaik, S. Mishra, A. V. Thomas, N. Shallum, K. Ghosh, and A. Bhattacharya, "NyayaRAG: Realistic legal judgment prediction with RAG under the Indian common law system," arXiv:2508.00709 (AACL-IJCNLP 2025), 2025.

[28] Supreme Court of India, "SUVAS: Supreme Court Vidhik Anuvaad

Software for multilingual judicial translation," Official Portal, Supreme Court of India, 2021.
[29] ——, "SUPACE: Supreme Court Portal for Assistance in Court's Efficiency," Official Portal, Supreme Court of India, 2021.
[30] Dattam Labs, "Indian High Court judgments: dataset documentation," https://github.com/vanga/indian-high-court-judgments/blob/main/opendata/docs/dataset.md, accessed Sep. 25, 2026.
[31] Supreme Court of India, "Notice: Launch and implementation of neutral citation system at Supreme Court of India in a phased manner," Jul. 2023, notice dated July 6, 2023.
[32] ——, "Circular: Implementation of phase-II of the neutral citation system," Sep. 2023, circular dated Sep. 27, 2023; covers reported judgments of 30.01.1950–31.12.2013 published in the Supreme Court Reports.
[33] K. Spärck Jones, "A statistical interpretation of term specificity and its application in retrieval," *Journal of Documentation*, vol. 28, no. 1, pp. 11–21, 1972.
[34] Python Software Foundation, "difflib — helpers for computing deltas (SequenceMatcher)," Python 3.13 documentation, https://docs.python.org/3/library/difflib.html.
[35] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.
[36] J. Chen, S. Xiao, P. Zhang, K. Luo, D. Lian, and Z. Liu, "M3-Embedding: Multi-linguality, multi-functionality, multi-granularity text embeddings through self-knowledge distillation," arXiv:2402.03216, 2024.
[37] Ollama, "Ollama," https://github.com/ollama/ollama, version 0.34.4.
[38] A. Grattafiori *et al.*, "The Llama 3 herd of models," arXiv:2407.21783, 2024.
[39] Qwen Team, "Qwen3 technical report," arXiv:2505.09388, 2025.
[40] Gemma Team, "Gemma 3 technical report," arXiv:2503.19786, 2025.
[41] S. Bai *et al.*, "Qwen2.5-VL technical report," arXiv:2502.13923, 2025.
[42] OpenRouter, "OpenRouter: a unified interface for language models," https://openrouter.ai, accessed Oct. 2026.
[43] Mozilla, "PDF.js," https://mozilla.github.io/pdf.js/, accessed Oct. 2026.
[44] R. Smith, "An overview of the Tesseract OCR engine," in *Proceedings of the Ninth International Conference on Document Analysis and Recognition (ICDAR)*, 2007, pp. 629–633.
[45] Government of India, "The limitation act, 1963 (act 36 of 1963)," 1963.
[46] Supreme Court of India, "Supreme court rules, 2013," 2013.